

RISC-V Single-Cycle Processor

Design & Implementation Report

Verilog HDL Implementation on FPGA

Computer Architecture

April 2026

Syed Sualeh Abbas, Mohsin Mujtaba, Zaid Naqvi

1. Introduction

This report documents the design and implementation of a 32-bit RISC-V single-cycle processor written in Verilog HDL. The processor implements a subset of the RV32I base integer instruction set and is intended for deployment on an FPGA development board. It supports MMIO-based I/O through on-board switches and LEDs, enabling interactive programs to run directly on hardware.

The design follows a classic five-stage data path structure: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB), implemented as a single-cycle architecture.

2. Supported Instructions

The processor supports the following RISC-V RV32I instruction types:

Instruction Type	Opcode	Examples	Description
R-type	0110011	ADD, SUB, AND, OR, XOR, SLL, SRL, SLT	Register-to-register arithmetic and logic
I-type ALU	0010011	ADDI, SLTI, SLLI, SRLI	Immediate arithmetic and logic
Load (LW)	0000011	LW	Load 32-bit word from memory
Store (SW)	0100011	SW	Store 32-bit word to memory
Branch	1100011	BEQ, BNE	Conditional branches on equality/inequality
JAL	1101111	JAL	Jump and link — unconditional jump, saves PC+4

3. Top-Level Architecture

The TopLevelProcessor module is the top-level entity that instantiates and connects all sub-modules. It exposes a rich set of debug output ports to allow real-time inspection of every internal signal during simulation and on-hardware debugging.

3.1 Datapath Overview

The five-stage single-cycle datapath flows as follows:

- IF: The Program Counter (PC) drives the Instruction Memory. PC+4 is computed by PCAdder for the default sequential flow.
- ID: The fetched instruction is decoded by MainControl and ALUControl. The RegisterFile is read and the ImmGen sign-extends the immediate.
- EX: The ALU performs the required computation. ALU_Mux selects between the register value (rd2) and the sign-extended immediate as the second operand.
- MEM: AddressDecoder routes memory accesses to DataMemory, the LED peripheral, or the Switch peripheral based on the upper address bits.
- WB: A 3-way mux (Mux32_3way) selects the write-back value: ALU result, memory read data, or PC+4 (used by JAL).

3.2 PC Selection Logic

The next PC is determined by the BranchLogic module and the PC Mux:

- pc_sel = 0 → PC takes the value PC+4 (normal sequential execution).
- pc_sel = 1 → PC takes the value PC+imm (branch taken or JAL).

The BranchAdder computes PC + imm_ext for all branch targets and the JAL destination, sharing hardware between jumps and branches.

4. Module Descriptions

4.1 ProgramCounter

The ProgramCounter module holds the current instruction address. On every rising clock edge it samples the pc_in value (the next PC computed externally) into the pc_out register. An asynchronous active-high reset drives pc_out to 0x00000000, restarting execution from the first instruction.

Port	Direction	Width	Description
clk	Input	1-bit	System clock
reset	Input	1-bit	Asynchronous reset (active-high)
pc_in	Input	32-bit	Next PC value from PC mux
pc_out	Output	32-bit	Current PC (registered)

4.2 PCAdder

A purely combinational adder that computes $PC + 4$. This gives the default next instruction address for sequential execution. Its output (`pc_plus4`) is also routed to the write-back mux so that JAL can save the return address ($PC+4$) into the destination register.

4.3 BranchAdder

Computes $PC + \text{imm_ext}$ combinationaly. This value serves as the target address for all taken branches (BEQ, BNE) and for the JAL instruction. Because JAL and branches share the same offset encoding relative to the current PC, one adder serves both instruction types.

4.4 BranchLogic

This module centralizes all branch and jump decisions into a single combinational block, producing the `pc_sel` signal that drives the PC mux. The decision tree is:

- **Unconditional jump:** JAL (`jal=1`)
- **Taken when `alu_zero=1` (subtraction result is zero):** BEQ (`branch=1, funct3=000`)
- **Taken when `alu_zero=0` (subtraction result is non-zero):** BNE (`branch=1, funct3=001`)
- **`pc_sel=0`, no branch taken:** All other cases

BNE support was a key addition to this design. Rather than hardwiring BEQ behaviour, a case statement on `funct3` allows easy extension to BLT, BGE, and other branch variants in the future.

4.5 InstructionMemory

A 64-entry, 32-bit wide read-only ROM initialized using an initial block. Word-aligned access is achieved by indexing with `address[7:2]` (dividing the byte address by 4). During reset the module outputs a 32-bit zero rather than a potentially invalid instruction.

The ROM is pre-loaded with a test program that exercises the full instruction set. The program reads from the on-board switches, displays the value on the LEDs, counts down through a software delay loop, decrements the switch value, and repeats until it reaches zero, then halts.

For Example: FSM COUNTER PROGRAM FOR PART A:

Address	Instruction	Assembly	Purpose
0x00	0x20000093	<code>addi x1, x0, 0x200</code>	<code>x1</code> = Switch MMIO base address
0x04	0x10000113	<code>addi x2, x0, 0x100</code>	<code>x2</code> = LED MMIO base address
0x08	0x0000a183	<code>lw x3, 0(x1)</code>	READ_SW: load switch value
0x0C	0xfe018ee3	<code>beq x3, x0, -4</code>	Spin until switch != 0
0x10	0x00312023	<code>sw x3, 0(x2)</code>	DISPLAY: write <code>x3</code> to LEDs
0x14	0x0007a313	<code>addi x6, x0, 122</code>	Load delay seed
0x18	0x00c31313	<code>slli x6, x6, 12</code>	Scale to ~500 000 iterations
0x1C	0xff30313	<code>addi x6, x6, -1</code>	D_INNER: decrement counter

Address	Instruction	Assembly	Purpose
0x20	0xfe031ce3	bne x6, x0, -8	Loop until counter = 0 (BNE test)
0x24	0xffff18183	addi x3, x3, -1	Decrement display value
0x28	0xfe019ce3	bne x3, x0, -24	Return to DISPLAY if x3 != 0 (BNE test)
0x2C	0x00012023	sw x0, 0(x2)	Turn off LEDs
0x30	0x0000006f	jal x0, 0	Halt — infinite self-loop (JAL test)

4.6 MainControl

The MainControl module decodes the 7-bit opcode and drives all control signals across the datapath. The module uses a combinational always block with safe defaults (all zeros) before a case statement, ensuring no latches are inferred.

Opcode	Instruction	reg_write	alu_src	mem_read	mem_write	branch	jal	wb_sel	alu_op
0110011	R-type	1	0	0	0	0	0	00	10
0010011	I-type ALU	1	1	0	0	0	0	00	10
0000011	Load (LW)	1	1	1	0	0	0	01	00
0100011	Store (SW)	0	1	0	1	0	0	00	00
1100011	Branch	0	0	0	0	1	0	00	01
1101111	JAL	1	1	0	0	0	1	10	00

The 2-bit wb_sel signal was introduced to replace the single-bit mem_to_reg, enabling a three-way write-back selection: 00 = ALU result, 01 = memory read data, 10 = PC+4 (required for JAL to save the return address).

4.7 ALUControl

ALUControl translates the 2-bit alu_op signal (from MainControl) together with funct3 and funct7 fields into a 4-bit alu_ctrl code consumed by the ALU. The three-level decode scheme mirrors the standard RISC-V control hierarchy:

alu_op	Source	alu_ctrl	ALU Operation
00	Load / Store	0000	ADD (address computation)
01	Branch	0001	SUB (equality comparison via zero flag)
10 / funct3=000	R-type / I-type	0000 or 0001	ADD or SUB (funct7 selects)
10 / funct3=001	SLLI / SLL	0101	Logical left shift

alu_op	Source	alu_ctrl	ALU Operation
10 / funct3=010	SLTI / SLT	0111	Signed less-than (new)
10 / funct3=100	XOR	0100	Bitwise XOR
10 / funct3=101	SRLI / SRL	0110	Logical right shift
10 / funct3=110	OR	0011	Bitwise OR
10 / funct3=111	AND	0010	Bitwise AND

4.8 RegisterFile

A 32-register, 32-bit wide synchronous register file. Key design properties:

- x0 is hardwired to zero: reads always return 0, writes are silently discarded.
- Two independent asynchronous read ports (rs1, rs2) for simultaneous operand fetch.
- One synchronous write port (rd) triggered on the rising clock edge when reg_write=1.
- Asynchronous reset clears all registers to zero in a single cycle.

4.9 ImmGen (Immediate Generator)

The ImmGen module reconstructs and sign-extends the immediate field from the instruction encoding. Different instruction formats scatter the immediate bits across non-contiguous fields; ImmGen re-assembles them correctly for each type:

Format	Instruction Type	Immediate Bits
I-type	ADDI, LW, SLTI, SLLI, etc.	inst[31:20], sign-extended to 32 bits
S-type	SW	inst[31:25] ++ inst[11:7], sign-extended
B-type	BEQ, BNE	inst[31], inst[7], inst[30:25], inst[11:8], 0 — sign-extended (byte offset)
J-type	JAL	inst[31], inst[19:12], inst[20], inst[30:21], 0 — sign-extended (byte offset)

Both B-type and J-type immediates are implicitly multiplied by 2 (appended with a '0' LSB) because all RISC-V instructions are 4-byte aligned, making the LSB of any branch/jump offset always zero.

4.10 ALU

The Arithmetic Logic Unit is an 8-operation combinational block. The 4-bit `alu_ctrl` input selects the operation. A zero flag is derived from the result and used by BranchLogic for conditional branches.

alu_ctrl	Operation	Expression
0000	ADD	$A + B$
0001	SUB	$A - B$
0010	AND	$A \& B$
0011	OR	$A B$
0100	XOR	$A \wedge B$
0101	SLL (Shift Left Logical)	$A \ll B[4:0]$
0110	SRL (Shift Right Logical)	$A \gg B[4:0]$
0111	SLT (Set Less Than)	$\text{signed}(A) < \text{signed}(B) ? 1 : 0$

4.11 Mux32 (2-Input Multiplexer)

A generic 32-bit, 2-input multiplexer used in two places: the ALU operand-B mux (selects between `rd2` and `imm_ext`) and the PC mux (selects between `PC+4` and `PC+branch/jump target`).

A 32-bit, 3-input multiplexer used exclusively in the Write-Back stage. The 2-bit `sel` input chooses among: ALU result (00), memory read data (01), or `PC+4` (10). The `PC+4` path is what allows JAL to write the return address into the destination register without any additional hardware beyond this wider mux.

4.13 AddressDecoder

The AddressDecoder acts as an MMIO bus fabric. It decodes the upper address bits [9:8] to route memory transactions to one of three sub-modules:

Address Bits [9:8]	Peripheral	Base Address	Access Type
00	DataMemory	0x000	Read / Write
01	LEDs	0x100	Write only
10	Switches	0x200	Read only

Read data is selected combinationaly: if a data memory or switch access is active, the corresponding `read_data` is forwarded; otherwise, `32'd0` is returned. This ensures X-propagation does not reach the register file during write-only or non-memory instructions.

4.14 DataMemory

A 512-entry, 32-bit synchronous RAM. Writes are clocked; reads are asynchronous (combinational). The address is word-aligned using bits [10:2] (ignoring the two LSBs). Asynchronous reset zeroes the entire memory array.

4.15 LEDs

A 16-bit output register connected to the physical LED bank of the FPGA. It captures `write_data[15:0]` on the rising clock edge when `write_enable` is asserted. The `leds_out` wire drives the top-level output port directly and can be connected to on-board LEDs or a testbench monitor.

4.16 Switches

A 16-bit input sampler for the physical slide-switches of the FPGA. The raw `switches_in` signal is registered (sampled) on each rising clock edge to avoid metastability. The registered value is presented as a zero-extended 32-bit word on `read_data` when `read_enable` is asserted.

5. Design Extensions and Key Changes

5.1 BNE Support

BNE (Branch if Not Equal) was added by extending `BranchLogic` with a case statement on `funct3`. For BEQ the branch is taken when `alu_zero=1`; for BNE it is taken when `alu_zero=0`. Both use the same subtract-and-check-zero mechanism in the ALU.

5.2 JAL Instruction

JAL (Jump and Link) required three coordinated changes across modules:

- **MainControl:** A new `jal` output signal is asserted for opcode 1101111. `wb_sel` is set to 2'b10 so the write-back mux selects PC+4 (the return address). `reg_write=1` ensures the return address is committed to rd.
- **BranchLogic:** When `jal=1`, `pc_sel` is unconditionally set to 1, directing the PC mux to the BranchAdder output (PC + J-type immediate).
- **Mux32_3way (WB mux):** Upgraded from a 1-bit-select 2-way mux to a 2-bit-select 3-way mux, adding the PC+4 input required by JAL.

5.3 SLTI Instruction

SLTI (Set Less Than Immediate) was enabled by adding `funct3=010` to the ALUControl decode table, mapping it to `alu_ctrl=4'b0111` (the SLT operation already implemented in the ALU). No changes to the ALU itself were required.

6. Test Program Behavior

The ROM-resident test program follows the procedure written below: Which includes part A and part C of the project:

PROGRAM 1: FSM COUNTER

(Code done in Lab 10)

- Initialise: Load switch and LED MMIO addresses into x1 and x2 (ADDI).
- Read Switches: Poll the switch MMIO register until a non-zero value is read (LW + BEQ loop).
- Display: Write the switch value to the LED MMIO register (SW).
- Delay Loop: Load a large counter (~500 000), shift it left 12 places (SLLI), then count down to zero using BNE — producing a visible human-scale delay on hardware.
- Decrement: Subtract 1 from the displayed value (ADDI with -1).
- Repeat or End: If the value is non-zero, branch back to Display (BNE); otherwise turn off LEDs (SW x0) and halt via infinite JAL self-loop.

This program tests BEQ, BNE (in both the delay and decrement loops), ADDI, SLLI, LW, SW, and JAL, covering all major datapath paths including the new JAL write-back and BNE branch logic.

PROGRAM 2: 3- New type of Instructions on Processor

- The program begins by loading fixed values into registers, including the switch address, LED address, and a constant value of 16. This constant acts as a safety limit against which the user input will be compared.
- The processor then reads a value from the switches into register x6. This value represents the user's input and is the main variable used in the program.
- The first newly implemented instruction is SLTI (Set Less Than Immediate). This instruction compares the constant value 16 (stored in x5) with the input value in x6. If the input is greater than 16, the result of the comparison sets register x7 to 1; otherwise, it sets it to 0. This allows the processor to make a decision based on a numerical condition, which was not possible with only basic arithmetic instructions.
- The second key addition is the use of enhanced branch logic through BNE (Branch if Not Equal). In the loop, the processor checks whether the value in x6 has reached zero. If it has not, the program branches back and continues the loop. This enables iterative behavior, allowing repeated execution of instructions until a condition is met.
- The third newly implemented instruction is JAL (Jump and Link). This is used in two places. First, it allows the processor to jump to an error-handling section if the input exceeds the safe limit. Second, it is used to halt execution by creating an infinite loop. This instruction is important because it enables unconditional jumps, which were not supported in the basic design.
- If the input value is within the safe range, the processor proceeds to perform the arithmetic operation. It initializes a sum register and repeatedly adds the current value of x6 to the sum while decrementing x6 in each iteration.
- The loop continues until x6 becomes zero, at which point the total sum has been computed. This demonstrates how the processor combines arithmetic operations with branching to perform a complete algorithm.
- Once the computation is complete, the result is written to the LED output so that the user can observe it.

INSTRUCTION MEMORY: BNE, JAL, SLTI

```
// --- Setup ---
rom[0] = 32'h20000093; // addi x1, x0, 0x200 | x1 = Switch Address
rom[1] = 32'h10000113; // addi x2, x0, 0x100 | x2 = LED Address
rom[2] = 32'h01000293; // addi x5, x0, 16 | x5 = 16 (Our "Safe Limit")

// --- Step 1: Input & Comparison (SLT) ---
// START:
rom[3] = 32'h0000a303; // lw x6, 0(x1) | x6 = Read Switches
rom[4] = 32'h0062a393; // slti x7, x5, x6 | IF 16 < switches, x7 = 1, ELSE 0
rom[5] = 32'h00038863; // beq x7, x0, 16 | IF x7 == 0 (Safe), jump to MATH

// --- Step 2: JAL (Jump if Limit Exceeded) ---
rom[6] = 32'h01c0006f; // jal x0, 28 | JUMP to ERROR_STATE (x0 = discard link)

// --- Step 3: Math Logic (The Series) ---
// MATH:
rom[7] = 32'h00000233; // add x4, x0, x0 | x4 (Sum) = 0
// LOOP:
rom[8] = 32'h00620233; // add x4, x4, x6 | Sum = Sum + x6
rom[9] = 32'hfff30313; // addi x6, x6, -1 | x6 = x6 - 1
rom[10] = 32'hfe031ce3; // bne x6, x0, -8 | IF x6 != 0, GOTO LOOP (BNE TEST)

// --- Step 4: Output ---
rom[11] = 32'h00412023; // sw x4, 0(x2) | Display Sum on LEDs
rom[12] = 32'h0000006f; // jal x0, 0 | DONE: Spin here

// --- Step 5: Error State (Flashing) ---
// ERROR_STATE:
rom[13] = 32'hfff00393; // addi x7, x0, -1 | x7 = 0xFFFF
rom[14] = 32'h00712023; // sw x7, 0(x2) | All LEDs ON
rom[15] = 32'h0000006f; // jal x0, 0 | Halt in Error
```

PROGRAM 3: ARITHMETIC SEQUENCE

- The processor first sets up the memory-mapped addresses by loading the switch address into register x1 and the LED address into register x2. These addresses are later used to read user input and display output.
- It then continuously reads from the switches until a non-zero value is detected. This first value is treated as the starting term of the sequence and is stored in register x3.
- After obtaining the starting value, the processor reads from the switches again to get the common difference. It again waits until a non-zero value is provided, ensuring that the sequence has a meaningful step size. This value is stored in register x5.
- The starting value is copied into register x4, which acts as the current term of the arithmetic sequence. This register will be updated in every iteration.
- In each cycle of the main loop, the current value in x4 is written to the LED output. This allows the user to observe the sequence as it evolves step by step.
- The processor then updates the current term by adding the common difference stored in x5 to x4. This produces the next term in the arithmetic sequence.
- A delay loop is introduced using register x6. The processor repeatedly decrements this register and checks whether it has reached zero. This delay slows down the execution so that changes on the LEDs are visible to the human eye.

- Once the delay completes, the processor jumps back to the beginning of the loop without any terminating condition. This creates a continuous sequence.
- As a result, the system generates an arithmetic progression where each new value is obtained by adding a fixed common difference to the previous value, and each term is displayed on the LEDs in real time.

INSTRUCTION MEMORY: Arithmetic Sequence:

```
initial begin
    // --- Setup ---
    rom[0] = 32'h20000093; // addi x1, x0, 0x200 | x1 = Switch address
    rom[1] = 32'h10000113; // addi x2, x0, 0x100 | x2 = LED address

    // --- Step 1: Read Starting Value (a) ---
    rom[2] = 32'h0000a183; // lw x3, 0(x1) | x3 = a (start)
    rom[3] = 32'hfe018ee3; // beq x3, x0, -4 | wait if a = 0

    // --- Step 2: Read Common Difference (d) ---
    rom[4] = 32'h0000a283; // lw x5, 0(x1) | x5 = d
    rom[5] = 32'hfe028ee3; // beq x5, x0, -4 | wait if d = 0

    // --- Step 3: Initialize Current Term ---
    rom[6] = 32'h00300233; // add x4, x0, x3 | x4 = a

    // --- Step 4: Arithmetic Sequence Loop ---
    // LOOP:
    rom[7] = 32'h00412023; // sw x4, 0(x2) | display current term

    rom[8] = 32'h00520233; // add x4, x4, x5 | x4 = x4 + d

    // --- Step 5: Visible Delay ---
    rom[9] = 32'h0007a313; // addi x6, x0, 122
    rom[10] = 32'h00c31313; // slli x6, x6, 12

    // D_INNER:
    rom[11] = 32'hfff30313; // addi x6, x6, -1
    rom[12] = 32'hfe031ce3; // bne x6, x0, -8

    // --- Step 6: Repeat Forever ---
    rom[13] = 32'hfe021ae3; // beq x0, x0, -24 | jump back to LOOP

    // END WITH NOP-
```

7. Conclusion

The RISC-V single-cycle processor successfully implements a functional RV32I subset capable of executing a real I/O program. The design is modular, with each hardware unit encapsulated in Verilog module, making it easy to verify, extend, and synthesise.

Key design decisions, such as the 3-way WB mux, the generalized BranchLogic module, and the shared BranchAdder for both branches and JAL, minimize hardware duplication while keeping the control logic easy to trace.

Signals List

Signal	Width	Description
pc_current	32-bit	Current instruction address (PC register output)
pc_next	32-bit	Next PC — output of PC mux; registered on next clock edge
pc_plus4	32-bit	Sequential next instruction (PC + 4)
pc_branch	32-bit	Branch/jump target (PC + sign-extended immediate)
pc_sel	1-bit	0 = sequential, 1 = branch/jump taken
instruction	32-bit	Full 32-bit instruction word from ROM
imm_ext	32-bit	Sign-extended immediate for current instruction
rd1 / rd2	32-bit	Register file read data for rs1 / rs2
alu_operand_b	32-bit	ALU B input — rd2 or imm_ext, selected by alu_src
alu_result	32-bit	ALU computed output
alu_zero	1-bit	1 when alu_result == 0 (used by branch logic)
alu_ctrl	4-bit	Selects ALU operation
alu_op	2-bit	Coarse ALU operation type from MainControl
reg_write	1-bit	Enables write to register file
mem_read	1-bit	Enables memory / MMIO read
mem_write	1-bit	Enables memory / MMIO write
alu_src	1-bit	Selects ALU operand B: 0 = rd2, 1 = imm_ext
mem_to_reg	1-bit	Legacy single-bit WB select (superseded by wb_sel)
wb_sel	2-bit	WB mux select: 00=ALU, 01=MEM, 10=PC+4
branch	1-bit	Indicates a conditional branch instruction
jal	1-bit	Indicates a JAL instruction
write_back_data	32-bit	Value written back to the register file
mmio_read_data	32-bit	Data returned from MMIO / data memory read